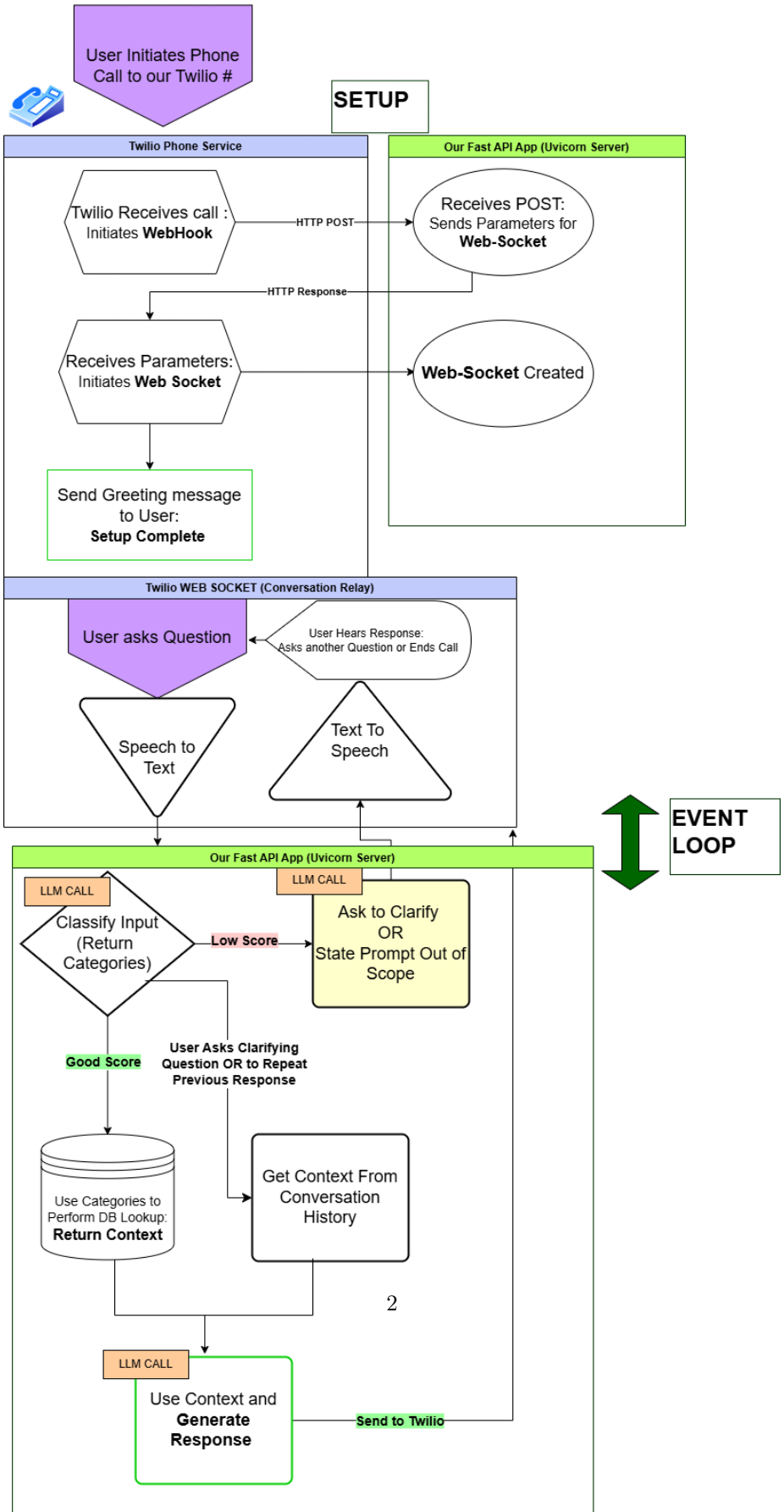
AI Admission Agent

Aidan Lee
Nicholai Platonoff

March 21, 2026

Brief Summary: Our project seeks to design a voice AI agent to help prospective students learn about the Northeastern admission process and answer any questions they may have. Users will interact with the system via a phone call. The system diagram is on the following page.

GitHub Click Here


User Initiates Phone Call to our Twilio #
SETUP
Twilio Phone Service
Our Fast API App (Uvicorn Server)
Twilio Receives call : Initiates WebHook
HTTP POST
Receives POST: Sends Parameters for Web-Socket
HTTP Response
Receives Parameters: Initiates Web Socket
Web-Socket Created
Send Greeting message to User: Setup Complete
Twilio WEB SOCKET (Conversation Relay)
User asks Question
User Hears Response: Asks another Question or Ends Call
Speech to Text
Text To Speech
EVENT LOOP
Our Fast API App (Uvicorn Server)
LLM CALL
Classify Input (Return Categories)
Low Score
LLM CALL
Ask to Clarify OR State Prompt Out of Scope
Good Score
User Asks Clarifying Question OR to Repeat Previous Response
Use Categories to Perform DB Lookup: Return Context
Get Context From Conversation History
LLM CALL
Use Context and Generate Response
Send to Twilio

1 Purpose of the Methodology

This methodology is effective in solving the problem because it enables real-time, natural communication between prospective students and the system. It gives them quick and precise access to the admission information that they want.

There are three critical parts of the system: SST, TTS models and the central agent. The Speech to Text (STT) component allows the system to understand the spoken input. The Text to Speech (SST) component enables the system to respond in a human like voice. An LLM acts as the central intelligent agent by retrieving info from the database and generating accurate, relevant answers to user inquiries. Unlike traditional rule based systems which rely on fixed rules and patterns to generate answers, this approach allows the system to provide dynamic context aware responses backed up by verified university information.

We will be comparing different models for two main reasons. The first model evaluation was performed after the dataset was created and tested how well it could categorize questions. The question categorization is to make DB retrievals. The cheapest model that met minimum needs was chosen for system prototyping and development. After the system is finished, more expensive and capable models will be substituted. We hope to see how much improvement is achieved as a function of cost.

2 Problem Statement

The problem addresses a Large Language Model (LLM) problem domain, specifically targeting conversational AI and question-answering over a domain specific knowledge base.

This project solves a real-world problem that better helps users and bolsters the current system. The objective of this project is to develop an AI telephone agent that helps prospective college applicants by answering their questions about Northeastern University. This eliminates the need for those applicants to spend endless hours looking through unofficial or official online sources without even knowing if they are actually accurate. The AI agent will consist of information coming from verified webpages.

This virtual agent system will allow students to gain quick access to the information desired in a swift manner, making the process more efficient and user-friendly. In addition, it reduces the burden on university staff by handling routine inquiries, allowing them to focus on more complex inquires individuals may have; the agent will escalate to human operator if user requests it or the system deems necessary.

Real Life Application:

1. Prospective student calls into the designated number.
2. AI agent answers and greets the caller.
3. The caller asks a question to the agent.
4. AI agent understands the caller's intent and digests the question.
5. Searches the university's database for relevant information.
6. AI agent formulates a clear and accurate response for the caller.
7. If needed, the agent will prompt if the caller needs more clarification.
8. If the question is not fully answered, a human representative will be dialed for assistance.
9. Information from the call will be logged in the database.

Related Work

1. Ben Epstein Voice Bot for Car Dealership Customer Service[1]

   Summary

   - This project serves as the main inspiration for our capstone. It is a voice AI agent demo. It is showcasing how a user can call the system with their phone and interact with the voice agent by asking questions about a car dealership. The application is customer service.
   - We plan to use much of the same architecture. Instead of developing an AI customer service agent for car sales, we would set up the system for admissions questions.

   Models/Approaches

   - Twilio used for telephony service: buy a phone number. The user will call this number to interact with the AI agent.
     - Automatically handles STT and TTS
   - Gemini LLM for main intelligence. Interface via official API call.
     - Gemma-3-4B open-source model deployed as backup in case API limit reached for Gemini.
   - Simple DB to store available car listings for the dealership.
   - BAML: extracts caller intent and questions. This then allows the LLM agent to perform "tool-calling". In this case, it is a function call to query from the DB.

   Main Results/Findings

- The agent responds in real-time and is able to be interrupted.
- The voice sounds realistic and not robotic. The system stores the user's phone number and questions asked: The system has memory of previous user interaction.

Limitations

- Data stored was discrete and simple, optimized for answering pointed questions (What is the price of your Toyota Camry?). For more open ended questions, the system will likely have difficulty. We hope to address this limitation with better DB or retrieval design.
- The system has no setup for load balancing or autoscaling for more concurrent users. We plan to deploy the code to Cloud system like GCP, to make use of the autoscaling features.
- No control over the TTS or STT models since they are operated under-the-hood by Twilio. We may investigate and use some different models instead.
- Additionally, to further distinguish from this work, we plan to add an admin portal interface to view all user call meta data. This would serve as a way to perform business analysis for the system. Examples would be to view most common questions, overall success/sentiment of the system, etc.

2. Mixed-Method (Cache and Retrieval) Augmented Generation System for Northeastern's Office of Global Studies) (Previous Student Paper)[2]

   Summary

   - Chatbot system that specifically aimed to help prospective international students answer questions about applying and visa issues.
   - The data was obtained from web scrapping relevant Northeastern websites.
   - Interface via Streamlit
   - It was a locally deployed system.

   Models/Approaches

   - Extensive web-scrapping approach using Scrapy and BeautifulSoup to obtain data from roughly 300 websites (Northeastern Office of Global Services).
   - Cache Augmentation Generation (CAG) and Retrieval Augmentation Generation (RAG) to obtain the relevant website data.
     - Multiple Embedding models used.
     - Multiple Document chunking strategies used.
     - Llama3.1, Microsoft Phi3.5, and MosaicML-mpt LLMs used for intelligent chat agent

Main Results/Findings

- Achieved roughly 76 percent accuracy on evaluation set of Q/A pairs.
- CAG proved that it had lower latency than RAG

Limitations

- A significant portion of development time went to web-scrapping and data cleaning: still creators state that they wish they collected more data.
- Small open source models were used locally. These were bottlenecks of performance and latency for the system. APIs or Cloud Hosting should have been considered.
- Only international students considered. Our project seeks to expand the scope of the project to any prospective students. This would include undergraduate admission questions.

3. Asterisk-AI-Voice-Agent[3]

Summary

- Flexible tool that sets up the infrastructure for creating AI voice agents. This includes the telephony service, TTS, STT models, and the central LLM. Users of the system would need to add their own data and retrieval logic.

Models/Approaches

- Options for system configurations (choice of how LLM is inferenced)
  - OpenAI/Google API calls for core LLM intelligence.
  - Fully local environment: self-hosted open-source models
  - Local environment + hybrid API calls
- SST Model for local deployment: 'Vosk'
- TTS Model for local deployment: 'Piper'
- Asterisk instead of Twilio for telephony services
- Web Interface to see basic dataflow.

Main Results

- Agent responds in real-time: user cannot detect latency
- Successful in providing multiple options for deployment (based on user's cost profile)

Limitations

- The Local deployment option has a noticeable performance degradation.

- Project does not showcase the effectiveness of a real scenario (demo only showed response to one simple test question).
  - It is not clear how the infrastructure designed will respond to certain questions and high user load.

4. A contemporary review on chatbots, AI-powered virtual conversational agents, ChatGPT: Applications, open challenges and future research directions[4]

   Summary

   - This research survey provides an in-depth overview of a wide variety of AI chatbots, which includes AI voice bots that leverage LLMs.
   - It examines the behavior, performance, evaluation metrics, and limitations of these AI systems.
   - Overall, we reference this work as a baseline for general scope and constraints that any LLM project will need to work within.

   Models/Approaches Used

   - N/A

   Main Results/Results

   - Emphasized the possibility for model hallucinations and the necessity of using model evaluation techniques.
   - One of the projects in the survey proved that it was possible to use a LLM for automating certain customer service messages.

   Limitations

   - Model hallucinations, prompt injection, input data drift, and input data skew are problems that need to be considered when creating any AI system.

3 Data Collection and Preparation

The primary data sources for this project come from the Northeastern University website. A few sources related to ranking originated from reputable ranking websites like US-news and times-higher-education.

Link to Data: Northeastern SharePoint Dataset

The dataset contains 46 rows and is structured as follows:

- `Group`
- `SubGroup`
- `Information Text`

- `Source(urls)`
- `Source_Type`
- `Lookup_Key`

The main features for the development of the Agent are the `Group`, `SubGroup`, `Information Text`, and `Lookup_Key` columns. To prepare the dataset for the LLM, the `Group` and `SubGroup` columns were created to organize and classify `Information Text` in the agent's database, enabling the system to retrieve relevant content based on the user query. The two columns together form the `Lookup_Key`, which the system uses to efficiently index and access the desired information. The `Information Text` column holds all the information that was extracted from multiple `Source(urls)`.The only processing was to add a pipe character "|" between urls so that they can easily be split since they are stored as a single string.

This dataset was primarily chosen because it was sourced from a credible online platform of Northeastern University. It contains information on admissions, finances, rankings, and other relevant details that prospective students typically seek before making decisions about their future at the university.

Example Entry: Group: Admissions/Applications, Subgroup: Tours, Source(urls): <two urls present>, Information Text: <all text from both urls> Lookup_Key: Admissions/Applications_Tours.

4 Selection of Machine Learning or LLM Models

4.1 Model Consideration

During the initial LLM selection process, multiple models were evaluated to determine which was best suited to handle the question categorization component of the system. This part is crucial because the question categorization component would need to generate the exact `Lookup_Key` that is present in the database given all the potential Groups and its SubGroups. Accurate categorization of the question ensures that the system can retrieve the correct information based on the input question provided. Additionally, the model chosen should be cheap to call to save on costs while we develop the product.

The LLM models used in the selection were: GPT-4o-mini, Gemini-3-Flash, Claude-Sonnet-4.5, DeepSeek-V3.2, and Qwen3. Each model was evaluated on its ability to correctly generate the right `Lookup_Key` for all possible combinations of the Group and SubGroup.

There will be a second round of evaluation of models after the product is finished. More expensive and higher quality models like Claude Opus 4.5, gpt5-latest and

Gemini-3.1-Pro will be used. It is assumed that there will be performance limits with the smaller and cheaper models, and performance gains of these larger models will be compared.

4.2 Final Model Selection Criteria

To determine the initial LLM to be used for prototyping and development: 50 questions and expected-key pairs were used. An example is ("What furniture comes with my dorm room?", Housing_Layout_Info). The code to run the testset is on the Github at

`/src/backend_test_deploy/backend_testing_local.py`

The model that has the highest number of test cases passed will be chosen.

Once the product is finished, the second round of evaluations will start with the larger LLMs. The models will be compared based on a score-cost ratio:

$$\frac{\#Tests\ Passed}{Cost\ Per\ Query}$$

The number of tests passed would simply be number of test cases passed and Cost-Per-Query would be the cost per Millions of input tokens. This test suite will be different than the previous: it will be evaluating all expected agent decisions instead of simple question categorization. It will be possible that a larger model may be marginally better in the number of tests passed, but its overall score worse because of the large price increase. Example:

$$\frac{50\ passed}{\$0.10} = 500 \quad , \frac{75\ passed}{\$0.50} = \quad 150$$

5 Model Development and Training

Since this project is LLM Engineering, no fine-tuning or training is necessary. The goal is not to predict classes or values, but rather utilize the natural language understanding and processing for a practical purpose. Achieving better results will come from improving code architecture and features, and simply choosing better models.

Training Process Not applicable.

Hyperparameter Tuning Although the model is not being trained, there are still parameters that can be varied that affect performance. We will primarily focus on varying the prompts and question categorization strategy which separates questions into groups or subgroups. There are two approaches for question categorization: allowing the groups and subgroups to be freely selected by the LLM or to have the groups and related subgroups directly tied. The latter

approach is safer since it is impossible to get an invalid group and subgroup combination; however, it requires significantly more input tokens in the prompt to ensure this.

6 Evaluation and Comparison

After the initial testing period, GPT-4o-mini was chosen to be used for prototyping and development. Although it scored the same as Gemini and worse than Claude, it had by far the cheapest cost per 1 million tokens at $0.15. The open source models were also cheap but had higher latency. Non perfect scores are acceptable because the system has the ability to ask a clarifying question back to the user. The scores are bellow.

- GPT-4o-mini: 43/50
- Gemini-3-Flash: 43/50
- Claude-Sonnet-4.5: 46/50
- DeepSeek-V3.2: 40/50
- Qwen3 41/50

As of the time of writing, the product is not yet finished and therefore we have not yet upgraded to the stronger models and evaluated them. Soon after this iteration, all features will be implemented, the test set will be finalized, and these models will be evaluated.

7 References:

[1] B. Epstein, "Ben-Epstein/voice-bot-gemini-baml: A voice bot using Twilio, Gemini, and baml," GitHub, https://github.com/Ben-Epstein/voice-bot-gemini-baml (accessed Jan. 30, 2026).

[2] Anonymous, Mixed-Method (Cache and Retrieval) Augmented Generation System for Northeastern's Office of Global Studies, `https://northeastern.sharepoint.com/:b:/s/DS5500Project_blqa3b/IQBpqqbDU1FWTbbRVO1lEXnDAVlqqr4xDwpPYi6VThJTGnc?e=kcylPQ`

[3] Hkjarral, "HKJARRAL/asterisk-ai-voice-agent: An open-source AI voice agent that integrates with asterisk/freepbx using Audiosocket/RTP Technology," GitHub, `https://github.com/hkjarral/Asterisk-AI-Voice-Agent/` (accessed Jan. 30, 2026).

[4] A. Casheekar, A. Lahiri, K. Rath, K. S. Prabhakar, and K. Srinivasan, "A contemporary review on Chatbots, AI-powered virtual conversational agents, chatgpt: Applications, open challenges and future research directions," Computer Science Review, vol. 52, p. 100632, May 2024. doi:10.1016/j.cosrev.2024.100632